DataFrameLib

Design Report

COP290 – Design Practices

Amar Sinha

April 2026

Contents

1 Overview **3**

2 Project Structure **3**

3 Type System **3**

4 Column Kernel Layer **4**

5 Expression System **4**
5.1 ExprNode Hierarchy . . . 5
5.2 Free-function Factories . . . 5
5.3 Operator Overloads on Expr . . . 5

6 EagerDataFrame **5**
6.1 I/O . . . 5
6.2 Transformations . . . 6

7 LazyDataFrame **6**
7.1 Plan Node Hierarchy . . . 7
7.2 Execution via collect() . . . 7
7.3 sink_csv / sink_parquet . . . 7
7.4 explain(path) . . . 7

8 Query Optimiser **7**
8.1 Constant Folding & Expression Simplification . . . 8
8.2 Constant Filter Elimination . . . 8
8.3 Predicate Pushdown . . . 9
8.4 Projection Pushdown . . . 11
8.5 Limit Pushdown . . . 11
8.6 Predicate Pushdown through GroupByAggregate . . . 12
8.7 Dead WithColumn Elimination . . . 13
8.8 Consecutive Projection Merging . . . 14
8.9 Join Predicate Splitting . . . 15

8.10 Filter Merge (Predicate Conjunction) 17
8.11 Multi-pass Convergence Strategy 17

9 Memory Management **17**

10 Example Usage **17**
10.1 Eager Mode 17
10.2 Lazy Mode with Optimizer 18

1 Overview

DataFrameLib is a C++ library for high-performance tabular data processing, modelled after the ideas behind Pandas and Polars. It provides two execution modes and a query optimiser:

- **EagerDataFrame** – immediate, materialised execution.
- **LazyDataFrame** – deferred execution via a computation DAG.
- **QueryOptimizer** – rule-based plan optimisation before materialisation.

All in-memory columnar storage and I/O (CSV and Parquet) is backed exclusively by **Apache Arrow**. The build system is CMake (C++20) with shared-library linkage against `Arrow::arrow_shared` and `Parquet::parquet_shared`.

2 Project Structure

```
.
 CMakelists.txt
 include/
    dataframelib/
        types.hpp          -- DType enum + Arrow type utilities
        column.hpp         -- Column value type + column-level ops
        expr.hpp           -- Expression DSL (Expr, ExprNode hierarchy)
        plan_node.hpp      -- Logical plan nodes (PlanNode hierarchy)
        eager_dataframe.hpp
        lazy_dataframe.hpp
        optimizer.hpp
 src/
     column.cpp             -- Arithmetic/comparison/string/null kernels
     expr.cpp               -- Expr factory functions
     plan_node.cpp          -- ToString() + BuildDotGraph()
     eager_dataframe.cpp    -- Eager execution engine
     lazy_dataframe.cpp     -- Lazy plan builder + collect() executor
     optimizer.cpp          -- QueryOptimizer + all rule passes
```

3 Type System

Supported column types are declared in `types.hpp`:

DType enum	Arrow type
`kInt32`	`arrow::int32()`
`kInt64`	`arrow::int64()`
`kFloat32`	`arrow::float32()`
`kFloat64`	`arrow::float64()`
`kString`	`arrow::utf8()`
`kBoolean`	`arrow::boolean()`

Type promotion rule. When a binary numeric operation mixes integer and floating-point operands the result type is resolved by `ResolvePromotedNumericType()` as: `double` if either side is `double`; otherwise `float` if either side is `float`; otherwise `int64` if either side is `int64`; otherwise `int32`. This covers the required rule `int` + `float` = `float`.

Missing values. The `Column::Make()` factory calls `RejectNaNInFloatingArrays()`, which scans floating-point arrays on construction and raises `arrow::Status::Invalid` if any NaN is found. Missing values must be represented as Arrow nulls only.

Incompatible operations (e.g. arithmetic on a string column) throw `arrow::Status::TypeError` immediately.

4 Column Kernel Layer

`column.cpp` / `column.hpp` define a set of free functions that operate on `Column` values:

- **Binary numeric:** `ApplyBinaryNumericOp` – addition, subtraction, multiplication, division, modulo with type-promotion and null propagation.
- **Unary absolute value:** `ApplyUnaryAbsOp` – dispatches per numeric type.
- **Comparison:** `ApplyComparisonOp` – supports all six relational operators across numeric, string and boolean columns. Any null operand produces a null result.
- **Boolean:** `ApplyBooleanBinaryOp`, `ApplyBooleanNotOp` – AND, OR, NOT on boolean columns.
- **Null predicates:** `ApplyNullPredicateOp` – `is_null` / `is_not_null`.
- **String unary:** `ApplyStringUnaryOp` – length (returns `int64`), to_lower, to_upper.
- **String predicates:** `ApplyStringPredicateOp` – contains, starts_with, ends_with (return `boolean`).

All kernels are hand-written (no Arrow compute kernel dependency) and propagate nulls element-wise.

5 Expression System

The expression DSL lives in `expr.hpp` / `expr.cpp`. An `Expr` is a thin wrapper around a `shared_ptr<ExprNode>`. Operator overloads build the expression tree without evaluating it, enabling the same `Expr` objects to be used in both eager and lazy contexts.

5.1 ExprNode Hierarchy

Node class	Kind	Description
`ColNode`	Col	Reference to a named column
`LitNode`	Lit	Scalar literal (Arrow Scalar)
`AliasNode`	Alias	Renames the output of a sub-expression
`BinaryNode`	Binary	Two-child arithmetic/comparison/boolean node
`UnaryNode`	Unary	Single-child (not, abs, is_null, is_not_null)
`StringOpNode`	StringOp	String function with optional string argument
`AggNode`	Agg	Aggregation function (sum, mean, count, min, max)

5.2 Free-function Factories

```
Expr col(std::string name);
Expr lit(int32_t value);    Expr lit(int64_t value);
Expr lit(float value);      Expr lit(double value);
Expr lit(bool value);       Expr lit(std::string value);
```

5.3 Operator Overloads on Expr

C++ syntax	Semantic
`e1 + e2` / `-` / `*` / `/` / `%`	Arithmetic (BinaryNode)
`e1 == e2` / `!=` / `<` / `<=` / `>` / `>=`	Comparison (BinaryNode)
`e1 & e2` / `e1 | e2` / `~e`	Boolean AND / OR / NOT
`e.abs()`	Absolute value
`e.is_null()` / `e.is_not_null()`	Null predicates
`e.length()` / `e.to_lower()` / `e.to_upper()`	String ops
`e.contains(s)` / `e.starts_with(s)` / `e.ends_with(s)`	String predicates
`e.sum()` / `e.mean()` / `e.count()` / `e.min()` / `e.max()`	Aggregations
`e.alias(name)`	Output rename

6 EagerDataFrame

`EagerDataFrame` stores data as a `std::vector<Column>` (in declaration order) plus an `arrow::Schema` and a row count. Every method materialises its result immediately and returns a new `EagerDataFrame` by value – no in-place mutation.

6.1 I/O

- **read_csv** – uses `arrow::csv::TableReader`; chunked arrays are flattened to contiguous arrays via `arrow::Concatenate`.
- **read_parquet** – uses `parquet::arrow::OpenFile` + `ReadTable`.
- **write_csv** / **write_parquet** – builds a temporary `arrow::Table` and delegates to `arrow::csv::WriteCSV` / `parquet::arrow::WriteTable`.

- **from_columns(map)** – accepts an `unordered_map<string, shared_ptr<Array>>`; validates equal lengths; sorts keys alphabetically for deterministic column order.

6.2 Transformations

select(columns) Picks a subset by name, preserving the requested order. Duplicate names throw immediately.

select(expressions) Evaluates each `Expr` via `EvaluateNodeToColumn`, which recursively dispatches on `ExprNode::Kind`. The output name defaults to the column reference name, the alias name if an `AliasNode` wraps the expression, or `expr_<i>` otherwise.

filter(predicate) Evaluates the predicate expression to a boolean column, then passes through `FilterArrayByMask` for each column. Null mask entries are treated as `false` (row excluded).

with_column(name, expr) Evaluates `expr`; if a column with the same name exists it is replaced in place; otherwise the new column is appended.

group_by(keys).aggregate(aggs) The group-by is implemented with a hash-based approach: `CellToKeyString` serialises each key column cell into a type-tagged string; groups are built into an `unordered_map` that preserves insertion order via a separate `group_order` vector. `AggregateArrayByGroups` then computes sum, mean, count, min, or max per group, returning `null` when all values in the group are null.

join(other, on, how) Supports inner, left, right, and outer joins. A hash index is built on the right table; the left table is probed. Null key values on either side are excluded from matching. Non-key columns from both sides are appended; duplicate non-key column names raise an error before execution begins.

sort(columns, ascending) A stable sort on a permutation index vector using a custom comparator that handles multi-key tie-breaking and null-last ordering.

head(n) Slices each column array via `arrow::Array::Slice`.

7 LazyDataFrame

`LazyDataFrame` holds a single `shared_ptr<PlanNode>`. Every transformation method creates a new plan node that wraps the current plan and returns a new `LazyDataFrame` by value. No data is read or processed until `collect()` is called.

7.1 Plan Node Hierarchy

Node class	Logical operation
`ScanCsvNode`	Leaf – CSV file path
`ScanParquetNode`	Leaf – Parquet file path
`SelectNamesNode`	Column projection by name
`SelectExprsNode`	Column projection by expression
`FilterNode`	Row predicate
`WithColumnNode`	Derived column addition / replacement
`GroupByAggregateNode`	Combined group-by + aggregation
`JoinNode`	Binary join (left + right inputs)
`SortNode`	Row ordering
`HeadNode`	Row-count limit

All nodes except `JoinNode` have a single primary `input()`. `JoinNode` additionally exposes `right_input()`, making the plan a DAG rather than a simple tree in the presence of joins.

7.2 Execution via collect()

`collect()` runs the `QueryOptimizer` on the plan and then calls `ExecutePlan(optimized_plan)`. The executor performs a post-order traversal of the plan DAG using an explicit stack (to avoid stack-overflow on deep plans), materialises each node into an `EagerDataFrame`, and stores results in an `unordered_map<const PlanNode*, EagerDataFrame>`. Each node retrieves its input(s) from this map and calls the corresponding `EagerDataFrame` method.

7.3 sink_csv / sink_parquet

These simply call `collect()` and forward to `EagerDataFrame::write_csv/write_parquet`.

7.4 explain(path)

`explain(path)` renders *two* PNG diagrams side-by-side:

1. `<base>_logical.png` – the unoptimised plan DAG.
2. `<base>_optimized.png` – the plan after `QueryOptimizer::optimize()`.

The DOT source is generated by `BuildDotGraph()` in `plan_node.cpp`, which assigns integer node IDs, emits box-shaped nodes labelled with `PlanNode::ToString()`, and draws directed edges from child to parent (data flows upward). The DOT file is then rendered to PNG by shelling out to `dot -Tpng` from Graphviz; the temporary `.dot` file is cleaned up afterwards.

8 Query Optimiser

`QueryOptimizer::optimize(root)` applies rules in multiple passes using `OptimizeTreeOnce()`, which traverses the plan bottom-up, rebuilds each node with optimised children, and applies all rule functions to the rebuilt node. Convergence is detected by a string fingerprint of the plan tree; the loop runs for at most 8 passes.

The following optimisations are implemented.

8.1 Constant Folding & Expression Simplification

Transformation

`OptimizeExpr(expr)` walks an expression tree bottom-up. If both children of a binary node or the child of a unary/string node reduce to literal values, the expression is evaluated at plan-construction time and replaced by a single `LitNode` carrying the result.

Additionally, the following algebraic identities are applied even when only one operand is a literal:

- $x + 0 \rightarrow x$ and $0 + x \rightarrow x$
- $x - 0 \rightarrow x$
- $x \times 1 \rightarrow x$ and $1 \times x \rightarrow x$
- $x/1 \rightarrow x$
- `true` & $e \rightarrow e$ and e & `true` $\rightarrow e$
- `false` | $e \rightarrow e$ and e | `false` $\rightarrow e$
- $\sim(\sim e) \rightarrow e$ (double negation elimination)

String and unary literal folds are also performed: e.g. `lit("hello").length()` → `lit(5)`, and `lit(true).is_null()` → `lit(false)`.

Correctness

All identities are standard algebraic equalities that hold over the nullable semantics defined for DataFrameLib. Division-by-zero and modulo-by-zero are guarded: the fold is skipped (returning the original expression) if the divisor is zero. The same applies to type mismatches (e.g. modulo on floating-point literals).

Example

```
col("salary") * lit(1) + lit(0)
// After constant folding + simplification:
col("salary")
```

Performance benefit

Eliminates redundant arithmetic and reduces the number of expression nodes that the execution engine must visit at runtime. For wide tables this avoids allocating and populating temporary intermediate arrays.

8.2 Constant Filter Elimination

Transformation

After `MakePlanWithOptimizedExprs` folds all literal sub-expressions, `ApplyConstantFilterEliminatio` inspects the predicate of a `FilterNode`. If the predicate is a boolean literal it is handled

as follows:

- **Predicate is `lit(true)`:** The filter node is removed entirely and replaced by its child – no row is ever excluded so the node is pure identity overhead.
- **Predicate is `lit(false)`:** The filter is replaced by `HeadNode(input, 0)`. This canonicalises "zero rows possible" into the existing `head` mechanism so that subsequent limit pushdown passes can propagate the zero-row signal as close to the scan as possible.

Correctness

$\sigma_{\texttt{true}}(R) = R$ and $\sigma_{\texttt{false}}(R) = \emptyset$ are set-algebra identities. The `false` rewrite is safe because `head(0)` on any relation always produces zero rows in the same schema, matching the semantics of an impossible filter. The pass runs immediately after constant folding so that predicates like `lit(1)==lit(1)` are already reduced to `lit(true)` before this rule fires.

Example

```
// Original (tautology predicate that folds to true)
scan_csv("employees.csv")
  .filter(lit(1) == lit(1))
  .select(...)

// After constant folding: lit(1)==lit(1) -> lit(true)
// After constant filter elimination: filter node removed
scan_csv("employees.csv")
  .select(...)
```

Performance benefit

Removes an entire filter evaluation pass – no boolean array is allocated, no mask scan is performed. For the `lit(false)` case, combined with limit pushdown it can eventually prevent reading the source file at all.

8.3 Predicate Pushdown

Transformation

`ApplyPredicatePushdown(node)` inspects a `FilterNode` and attempts to push it closer to the data source by examining the node immediately below it:

- **Filter over SelectNames:** If all columns referenced by the predicate are present in the projected set, the filter is pushed below the projection: `select(cols).filter(pred)` → `filter(pred).select(cols)`.
- **Filter over WithColumn:** If the predicate does not reference the newly derived column, the filter is pushed below the `with_column`: `with_column(c,e).filter(pred)` → `filter(pred).with_column(c,e)`.

- **Filter over Filter:** Two consecutive filters are merged into a single filter whose predicate is the AND of both. The merged predicate is itself passed through `OptimizeExpr`.
- **Filter over Sort:** The filter is pushed below the sort (sorting fewer rows is cheaper): `sort(...).filter(pred)` → `filter(pred).sort(...)`.
- **Filter over Join (inner / left / right):** If the predicate references only join-key columns, it is pushed into the appropriate side(s): both sides for inner joins, only the left for left joins, only the right for right joins. Full outer joins are left unchanged.

Correctness

Each case preserves the set of output rows:

- *Filter + SelectNames:* projection is a pure column subset; row identity is unchanged, so filter commutes.
- *Filter + WithColumn:* the derived column is not in the predicate, so the boolean mask is identical whether computed before or after the column is added.
- *Filter merge:* $\text{filter}(p_1)(\text{filter}(p_2)(t)) = \text{filter}(p_1 \wedge p_2)(t)$ by standard set-comprehension rules.
- *Filter + Sort:* sorting is a permutation; filtering a permutation gives the same set of rows as filtering first and then permuting.
- *Filter + Join on key columns (inner):* for an inner join the predicate on join keys can be applied to both sides before materialising the cross-product, because any row excluded by the predicate would not have contributed a matched pair.

Example

```
// Original plan
scan_parquet("employees.parquet")
  .sort({"salary"}, {true})
  .filter(col("dept") == lit("Engineering"))

// After predicate pushdown (filter pushed below sort)
scan_parquet("employees.parquet")
  .filter(col("dept") == lit("Engineering"))
  .sort({"salary"}, {true})
```

Performance benefit

Reduces the number of rows before expensive operations (sort is $O(n \log n)$; join is $O(n + m)$ average for hash-join). A filter that eliminates 90% of rows before a sort turns a million-row sort into a 100 000-row sort.

8.4 Projection Pushdown

Transformation

`ApplyProjectionPushdown(node)` inspects a `SelectNamesNode` or `SelectExprsNode` and attempts to reduce the number of columns read/carried at lower levels:

- **SelectNames over Filter:** Computes the union of the projected column set and the columns referenced by the filter predicate; inserts a `SelectNamesNode` below the filter to read only those columns. An idempotency guard avoids inserting a redundant projection if one already exists.
- **SelectNames over WithColumn:** If the derived column is not needed in the output, the entire `with_column` node is elided. If it is needed, a narrowing projection is inserted below `with_column` carrying only the columns required to compute the derived column plus the rest of the select list.
- **SelectExprs over Filter:** Analogous to the SelectNames case; the needed column set is the union of all columns referenced in the expression list and the filter predicate.

Correctness

Projection pushdown only removes columns that are not consumed by any downstream node. The union with filter/expression column sets ensures that no column that would affect computation is dropped. The idempotency guard (`IsExactSelectNames`) prevents infinite insertion of identical projection nodes.

Example

```
// Wide table: 20 columns; we only need "name" and "salary"
scan_csv("employees.csv")              // 20 columns
  .filter(col("dept") == lit("Eng")) // references "dept"
  .select({"name", "salary"})

// After projection pushdown
scan_csv("employees.csv")              // 20 columns
  .select({"dept", "name", "salary"}) // <-- pushed down
  .filter(col("dept") == lit("Eng"))
  .select({"name", "salary"})
```

Performance benefit

For wide Parquet files, reading only the required columns avoids deserialising and allocating memory for unused column data. For CSV the benefit is primarily reduced memory allocation and fewer array iterations in subsequent kernels.

8.5 Limit Pushdown

Transformation

`ApplyLimitPushdown(node)` handles a `HeadNode` and pushes the row limit earlier in the plan:

- **Head over Head:** Two consecutive limits are merged into a single `HeadNode` with $n = \min(n_1, n_2)$.
- **Head over SelectNames / SelectExprs:** The head is pushed below the projection: the limit is applied before column projection, reducing the number of rows on which expressions are evaluated.
- **Head over WithColumn:** The limit is pushed below the column derivation: fewer rows need the derived expression evaluated.

Correctness

Head merge: $\text{head}(n_1)(\text{head}(n_2)(t)) = \text{head}(\min(n_1, n_2))(t)$, since both limits take only the first k rows in the same order.

Head + SelectNames/SelectExprs/WithColumn: Column projections are pure transformations that do not reorder or drop rows. Therefore, applying the row limit before or after the projection yields the same rows; it is always cheaper to limit first.

Example

```
// Original
scan_csv("large.csv")
  .with_column("tax", col("salary") * lit(0.3))
  .head(10)

// After limit pushdown
scan_csv("large.csv")
  .head(10)                          // <-- pushed down
  .with_column("tax", col("salary") * lit(0.3))
```

Performance benefit

Avoids computing the derived column (or expression projection) for rows that will be discarded. For a 1 000 000-row table with `head(10)`, the `with_column` evaluates only 10 rows instead of 1 000 000.

8.6 Predicate Pushdown through GroupByAggregate

Transformation

When a `FilterNode` sits directly above a `GroupByAggregateNode` and every column referenced by the predicate is a group-key column, the filter is pushed below the aggregate:

$$\texttt{group_by}(K).\texttt{aggregate}(A).\texttt{filter}(p) \longrightarrow \texttt{filter}(p).\texttt{group_by}(K).\texttt{aggregate}(A)$$

The check is performed by `CollectColumnsFromExpr` on the predicate followed by a membership test against the key list. If any predicate column is not a group key (i.e. it is an aggregated output column such as `salary_mean`), the transformation is skipped to preserve correctness.

Correctness

A group-key column has the same value for every row in the group and survives into the aggregate output unchanged. Therefore a predicate that only tests key columns produces the same boolean result whether it is evaluated on the individual input rows or on the single representative row emitted per group. More formally: a group g with key value k is included in the output iff $p(k)$ is true; filtering input rows with $p(k)$ before grouping eliminates exactly those groups, so the set of surviving groups and their aggregate values are identical. Predicates that reference aggregated columns (e.g. `salary_mean > 80000`) cannot be pushed down because those values do not exist until after the aggregation completes.

Example

```
// Original plan (filter after aggregation)
scan_csv("employees.csv")
  .join(scan_csv("departments.csv"), {"dept_id"}, "inner")
  .group_by({"dept_name"})
  .aggregate({{"salary", "mean"}})
  .filter(col("dept_name") == lit("Engineering"))

// After predicate pushdown through GroupByAggregate
scan_csv("employees.csv")
  .join(scan_csv("departments.csv"), {"dept_id"}, "inner")
  .filter(col("dept_name") == lit("Engineering"))  // <--
    pushed down
  .group_by({"dept_name"})
  .aggregate({{"salary", "mean"}})
```

Performance benefit

Rows belonging to excluded groups never enter the hash-map construction phase of group_by. In the example above, only the Engineering rows ($\approx 40\%$ of the table) are hashed; HR and Finance rows are discarded cheaply by the filter scan. The saving scales with the selectivity of the key predicate and the cost of the aggregation kernel.

8.7 Dead WithColumn Elimination

Transformation

`ApplyDeadWithColumnElimination` removes a `WithColumnNode` whose derived column is provably never consumed by the immediately downstream node. Two structural patterns are handled:

- **SelectExprs above WithColumn:** The set of column names referenced across all output expressions is collected via `CollectColumnsFromExpr`. If the derived column name does not appear in that set, the `with_column` node is bypassed: `select_exprs(E).with_column` → `select_exprs(E)` with the input rewired to `with_column`'s own input.
- **GroupByAggregate above WithColumn:** If the derived column is neither a group key nor referenced in any aggregation expression, the `with_column` node is bypassed

similarly.

The `SelectNames + WithColumn` case (projection of a named column subset that does not include the derived column) is already handled by `ApplyProjectionPushdown`; this pass covers the complementary expression-select and aggregation cases.

Correctness

A `with_column` operation appends or replaces exactly one column and leaves all other columns and rows unchanged. If no downstream expression references the derived column, the output of the plan is identical whether or not the column was ever computed. The membership check against `CollectColumnsFromExpr` is conservative: it scans the entire expression tree of every downstream expression, so no reference can be missed.

Example

```
// Original
scan_csv("employees.csv")
  .with_column("double_salary", col("salary") * lit(2))
  .select(std::vector<Expr>{col("id").alias("eid"), col("name")
     })
  // "double_salary" not referenced in select expressions

// After Dead WithColumn Elimination
scan_csv("employees.csv")
  .select(std::vector<Expr>{col("id").alias("eid"), col("name")
     })
```

In the smoke test this fires together with Constant Filter Elimination in a single optimiser pass, reducing a four-node logical plan to two nodes:

```
scan → filter(1==1) → with_column("double_salary") → select_exprs
                              ⇓
                      scan → select_exprs
```

Performance benefit

Eliminates the expression evaluation kernel for the dead column entirely – no intermediate array is allocated or populated. For expressions involving expensive string operations or long arithmetic chains this saving can be substantial. The optimisation also reduces plan depth, which shortens the executor's dispatch loop.

8.8 Consecutive Projection Merging

Transformation

`ApplyProjectionMerging` inspects a `SelectNamesNode` and collapses it with a projection immediately below it:

- **SelectNames above SelectNames:** The outer (downstream) projection is rewired directly to the inner node's input, discarding the inner `SelectNamesNode` entirely.

Because the outer column list is already a subset of the inner one (if the plan is valid), no column safety check is required.

- **SelectNames above SelectExprs:** The expressions whose output names match the outer column list are filtered and reordered to match the outer column order, producing a single `SelectExprsNode`. If any required name cannot be resolved, the transformation is skipped.

Correctness

A projection is a pure column-subsetting operation. Applying two projections $\pi_A(\pi_B(R))$ where $A \subseteq B$ is equivalent to $\pi_A(R)$ by the standard projection absorption law. For the `SelectNames + SelectExprs` case, only the expressions whose declared output names are in the outer set are retained; all others are unreachable and their removal cannot affect the result.

Example

```
// Original (redundant wide projection followed by narrow one)
scan_csv("employees.csv")
  .select({"id", "name", "dept_id", "age", "salary"})
  .select({"id", "name", "salary"})

// After consecutive projection merging
scan_csv("employees.csv")
  .select({"id", "name", "salary"})
```

Performance benefit

Eliminates an entire projection pass – no intermediate column-subset array is allocated. More importantly it reduces plan depth, which shortens the dispatch loop in `ExecutePlan` and creates a shallower structure for other pushdown rules to traverse.

8.9 Join Predicate Splitting

Transformation

`ApplyJoinPredicateSplitting` fires when a `FilterNode` sits directly above an inner `JoinNode` and the predicate is a conjunction. The transformation proceeds as follows:

1. `SplitConjunction` recursively decomposes the predicate on `AND` boundaries into a flat list of clauses.
2. `TryGetOutputColumns` is called on each join input to determine its output column set. The function traverses `SelectNames`, `SelectExprs`, `Filter`, `Sort`, `Head`, `WithColumn`, `GroupByAggregate`, and `Join` nodes; it returns `nullopt` for bare scan nodes whose schema is unknown at plan time.
3. Each clause is classified: non-key columns that belong exclusively to the left schema are pushed to the left input; those belonging exclusively to the right schema are pushed to the right input; clauses that span both sides (or reference only join keys) remain above the join.

4. New `FilterNodes` are inserted into the respective inputs, and any remaining clauses are reassembled with `CombineClauses` above the rebuilt join.

The optimisation applies only to inner joins; outer, left, and right joins are left unchanged because null-padding semantics make independent pre-filtering incorrect.

Correctness

For an inner join, a row pair (L, R) appears in the output iff the join key matches *and* all predicates evaluate to true. If clause p_L references only columns from L and clause p_R references only columns from R, then:

$$\sigma_{p_L \wedge p_R}(L \bowtie R) = (\sigma_{p_L} L) \bowtie (\sigma_{p_R} R)$$

This follows because $p_L(L, R) = p_L(L)$ and $p_R(L, R) = p_R(R)$, so filtering can be applied to each side independently before the join is formed. The key-only check avoids pushing predicates on join-key columns (which would interact with the null-handling logic in the existing predicate pushdown rule).

Example

```
// Original: combined filter above join
scan_csv("employees.csv").select({"id","name","dept_id","age","salary"})
  .join(
    scan_csv("departments.csv").select({"dept_id","dept_name","location"}),
    {"dept_id"}, "inner")
  .filter(col("age") > lit(30) & col("dept_name").starts_with("E"))

// After join predicate splitting (age -> left, dept_name -> right)
scan_csv("employees.csv").select({...}).filter(col("age") > lit(30))
  .join(
    scan_csv("departments.csv").select({...}).filter(col("dept_name").starts_with("E")),
    {"dept_id"}, "inner")
```

Performance benefit

Both inputs to the join are filtered before the hash index is built and probed. In the example, only Engineering rows survive on the right (1 of 3 departments), and only employees older than 30 survive on the left (5 of 10). The hash index is built over 1 row instead of 3, and only 5 left rows are probed instead of 10 – a combined reduction of roughly 5× in join work on this dataset. The saving scales with selectivity and is particularly large when one side is highly selective.

8.10 Filter Merge (Predicate Conjunction)

This sub-transformation is embedded inside `ApplyPredicatePushdown` (the *Filter over Filter* case). Two consecutive `FilterNodes` are collapsed into one whose predicate is the AND of both. This reduces plan depth and allows subsequent passes to push the combined predicate further down.

8.11 Multi-pass Convergence Strategy

The optimizer applies `OptimizeTreeOnce` (a bottom-up traversal followed by one application of all rule functions per node) in a loop of at most 8 passes. After each pass, a string fingerprint of the plan tree is computed; if the fingerprint does not change the loop exits early. This handles chains of transformations where one pass creates an opportunity exploited by the next (e.g. predicate pushdown enabling projection pushdown).

9 Memory Management

The library uses the following C++ resource-management idioms throughout:

- **RAII via `shared_ptr`:** All heap-allocated objects (`ExprNode`, `PlanNode`, `arrow::Array`, `arrow::Schema`) are owned through `shared_ptr`. Reference-counted ownership ensures prompt cleanup as DataFrames and plan nodes go out of scope.
- **Move semantics:** `EagerDataFrame` holds its column vector and schema by value; factory functions and transformation methods use `std::move` to transfer ownership without unnecessary copying. Column vectors passed into `from_ordered_columns` are moved.
- **Value types for DataFrames:** Both `EagerDataFrame` and `LazyDataFrame` are copyable/movable value types; copying a `LazyDataFrame` is cheap (it copies a `shared_ptr`).
- **Arrow memory pool:** All Arrow array builders use the default Arrow memory pool, giving consistent allocation tracking.

No raw `new` / `delete` calls appear in the library code. Valgrind-clean operation follows from Arrow's own memory management.

10 Example Usage

10.1 Eager Mode

```cpp
#include "dataframelib/dataframelib.h"
using namespace dataframelib;

auto df = EagerDataFrame::read_parquet("data.parquet");
auto result = df.filter(col("age") > lit(30))
                .select({"name", "salary"});
result.write_csv("output.csv");
```

10.2 Lazy Mode with Optimizer

```
auto df = LazyDataFrame::scan_parquet("data.parquet");
auto result = df
    .filter(col("age") > lit(30))
    .select({"name", "salary"})
    .group_by({"dept"})
    .aggregate({{"salary", "mean"}});

result.explain("plan.png");   // renders logical + optimized DAGs
auto output = result.collect(); // triggers optimisation + execution
output.write_csv("summary.csv");
```